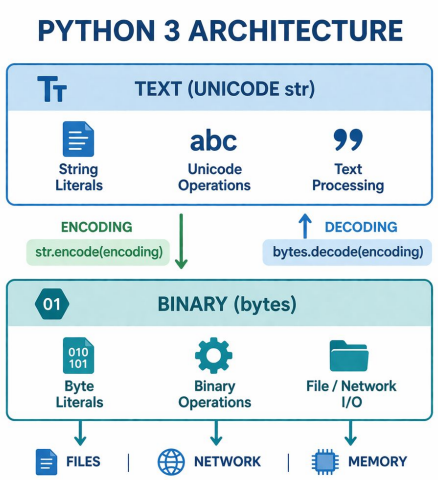
Python Programming Crash

Unicode and Byte Strings

Python 3 String Model

A modern and consistent approach to handling text and binary data in Python.

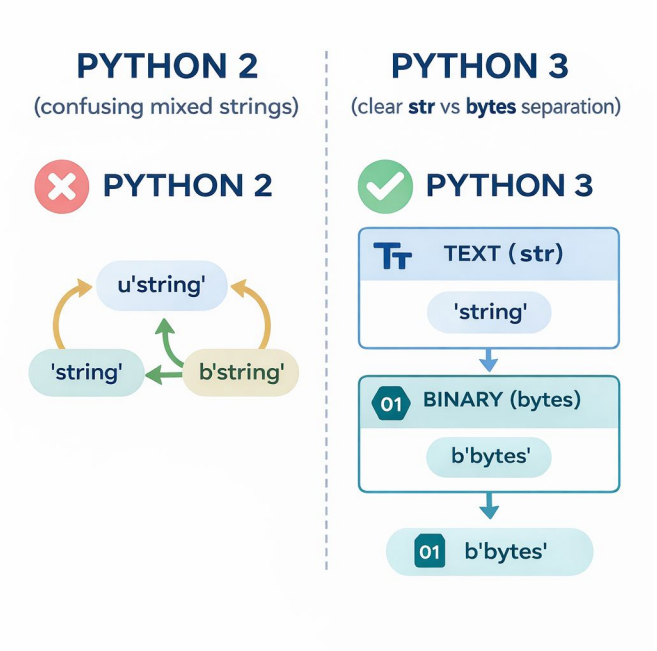
- Python 3 introduces a **clean separation of text and binary data**
- `str` is **Unicode by default**
- `bytes` represents **raw binary data**
- Eliminates confusion from Python 2 string handling
- Foundation for APIs, files, and data processing



Why Python 3 Changed Strings

Fixing ambiguity and making text handling consistent across systems.

- Python 2 mixed **text and binary in one type**
- Caused bugs with **encoding and international text**
- Python 3 enforces **explicit handling**
- Better support for **global languages (Unicode)**
- Reduces hidden errors in production systems



str vs bytes

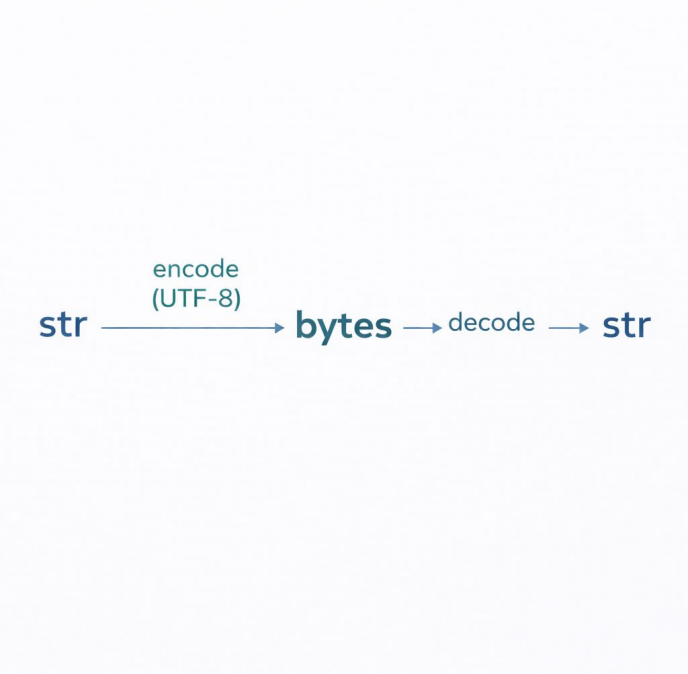
Understanding the difference between human-readable text and raw data.

- `str` = Unicode text (e.g., `"Hello"`)
- `bytes` = binary data (e.g., `b"Hello"`)
- `str` is used for **display and logic**
- `bytes` is used for **files, network, APIs**
- Cannot mix `str` and `bytes` directly

Encoding & Decoding

Transforming between text and binary using encoding standards.

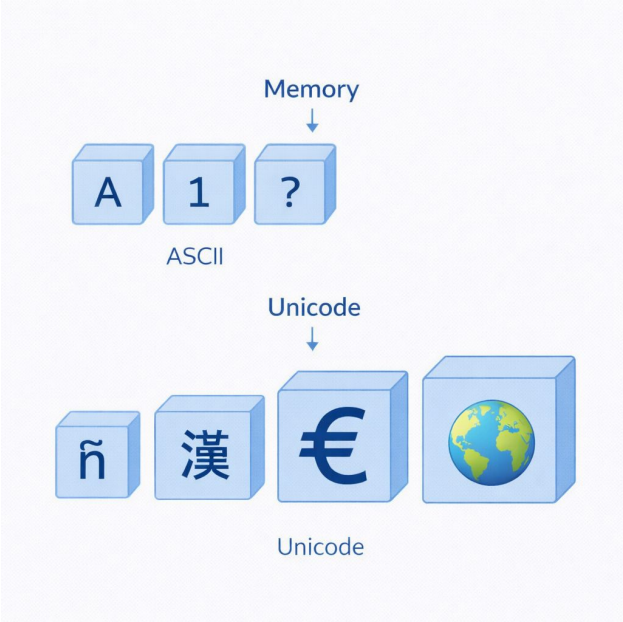
- Encoding: `str → bytes`
- Decoding: `bytes → str`
- UTF-8 is the **default and most common**
- ASCII is limited (only basic characters)
- Always specify encoding explicitly



Internal String Representation

How Python efficiently stores and manages Unicode strings in memory.

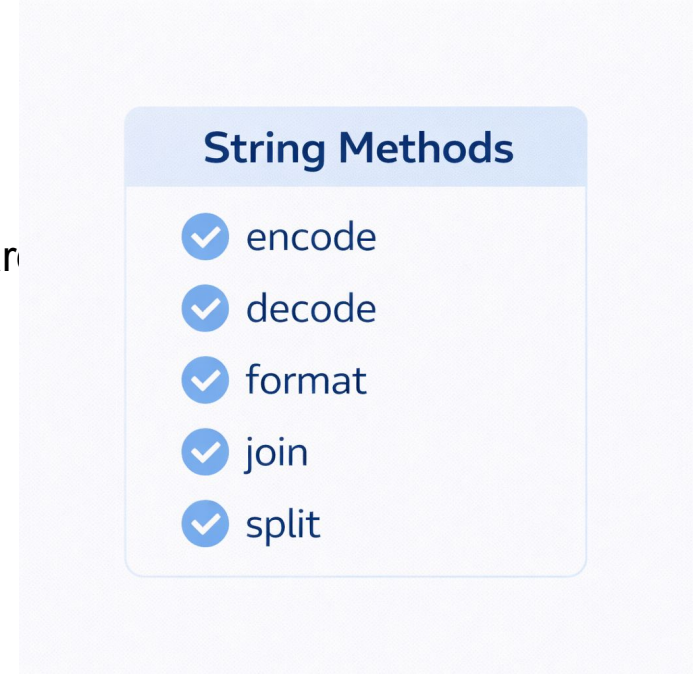
- Python uses **flexible internal representation**
- Optimizes memory based on character set
- Handles ASCII and Unicode efficiently
- Abstracted from developers (no manual control needed)
- Improves performance compared to Python 2



Python 3 String Tool Changes

Improved and more consistent string methods for modern development.

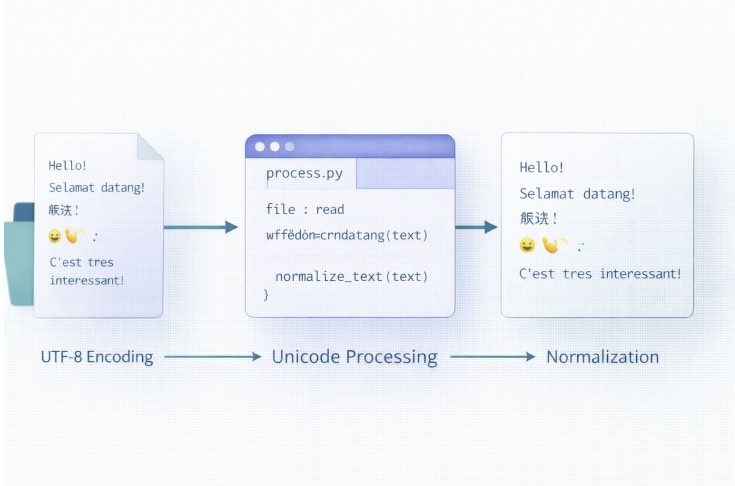
- Unified string methods across text handling
- Better Unicode support in all operations
- New methods like `.encode()` / `.decode()` standar
- Removal of Python 2 inconsistencies
- Cleaner and more predictable behavior



Lab 1: Unicode File Reader

Build a simple Python program that:

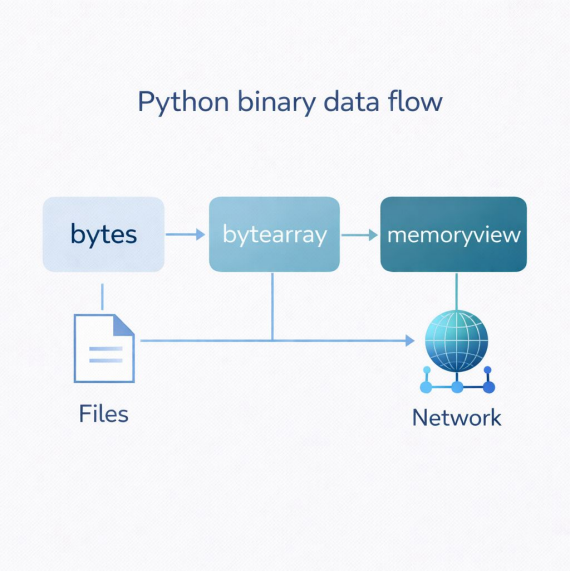
- reads a UTF-8 text file containing mixed languages
- handles encoding-related errors safely
- normalizes Unicode text for cleaner processing and comparison



Working with Bytes in Python

Handling raw binary data for files, networking, and performance-critical operations.

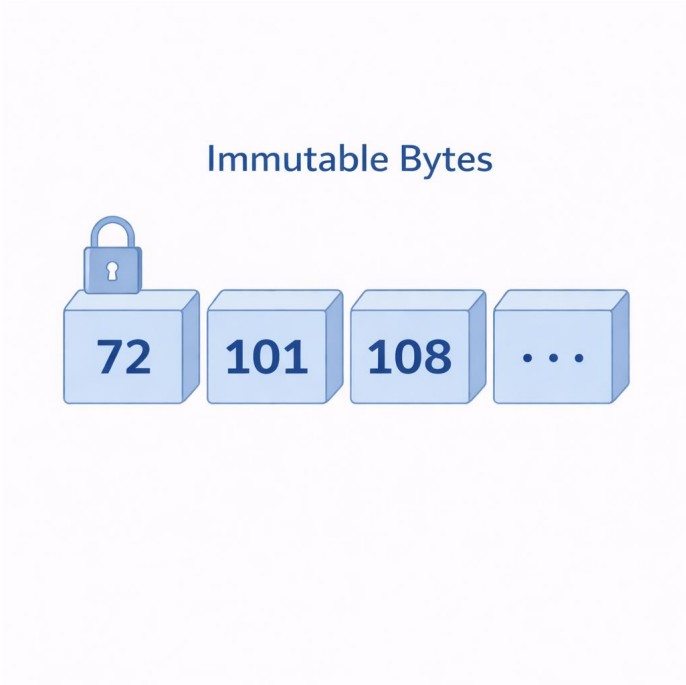
- Python provides multiple tools for **binary data handling**
- Core types: `bytes`, `bytearray`, `memoryview`
- Essential for **file I/O, APIs, and low-level operations**
- Works alongside `str` using encoding/decoding
- Enables efficient data processing



bytes Objects

Immutable sequences of raw binary data.

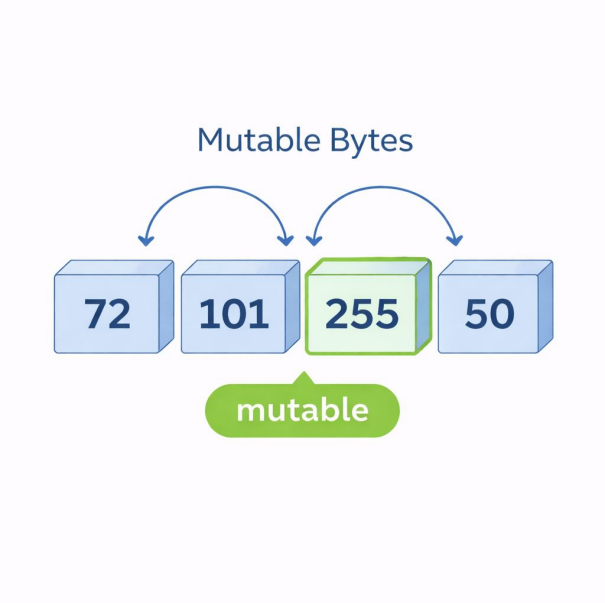
- Defined using `b""` syntax (e.g., `b"Hello"`)
- Each element is an integer (0–255)
- Immutable (cannot be changed after creation)
- Common in **networking and file reading**
- Supports indexing and slicing



bytearray Objects

Mutable version of bytes for dynamic binary data manipulation.

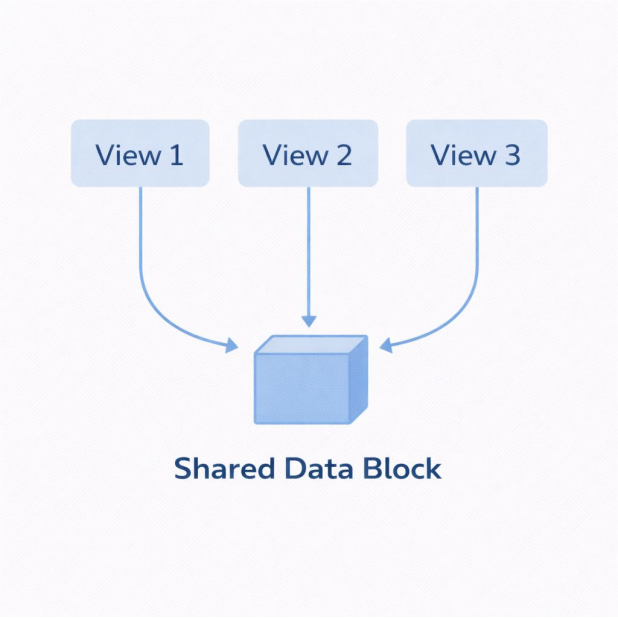
- Similar to `bytes` but **mutable**
- Can modify, append, or remove elements
- Useful for **buffers and data streams**
- Created using `bytearray()`
- More flexible but slightly less safe



Memory View Concepts

Efficient access to binary data without copying memory.

- `memoryview` allows **shared access to data**
- Avoids copying → improves performance
- Useful for **large datasets and streaming**
- Works with bytes-like objects
- Enables slicing without duplication



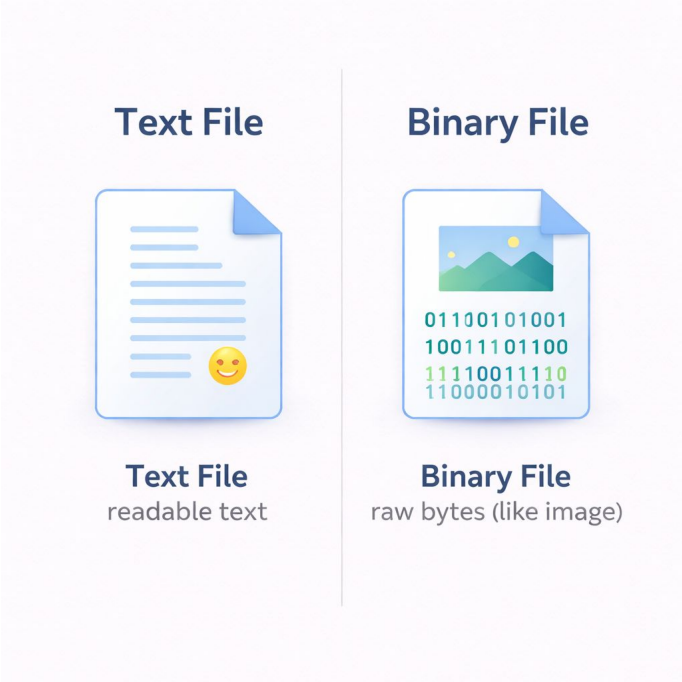
Text vs Binary Files

Understanding the difference between reading text and raw data from files.

- Text files use `str` (with encoding)
- Binary files use `bytes`
- Text mode: `"r"` / `"w"`
- Binary mode: `"rb"` / `"wb"`
- Choosing wrong mode causes errors

Example:

```
open("file.txt", "r")
open("image.png", "rb")
```



Unicode File Handling

Safely reading and writing text files using proper encoding.

- Always specify encoding (e.g., `utf-8`)
- Prevents errors with special characters
- Use `open(..., encoding="utf-8")`
- Important for global applications
- Avoid platform-specific encoding issues

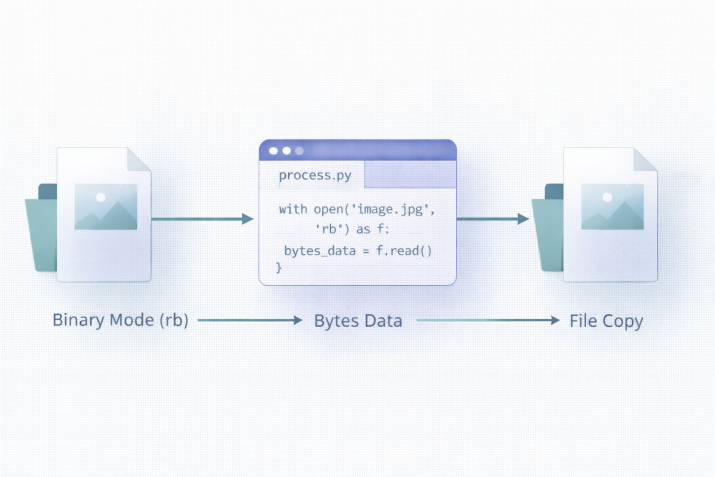
Example:

```
with open("file.txt", "r", encoding="utf-8") as f:
    content = f.read()
```

Lab 2 — Binary File Handling

Build a simple Python program that:

- reads an image file in binary mode
- converts the file content into bytes
- writes those bytes into a new file
- validates that the byte size matches the original file



Real-World String & Bytes Scenarios

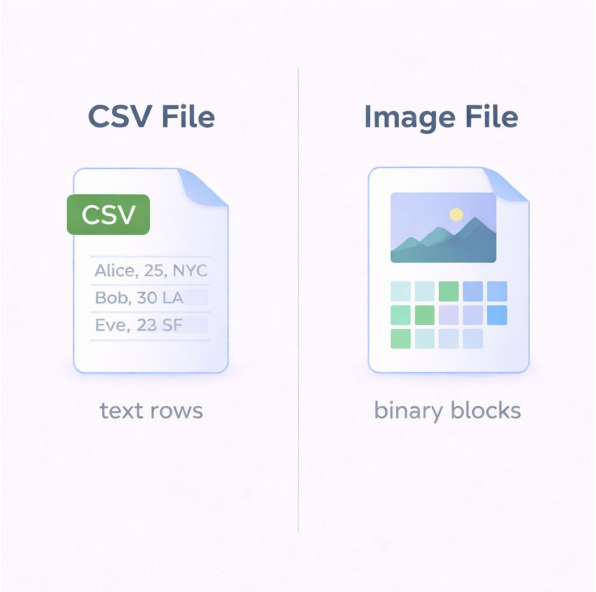
Applying Python string and byte concepts in real production use cases.

- Text vs binary handling appears in **every real system**
- APIs, files, uploads, and JSON all depend on encoding
- Small mistakes → **data corruption or crashes**
- Unicode handling is critical for **global applications**
- Practical understanding prevents production bugs

Reading CSV vs Binary Files

Choosing the correct mode for structured text vs raw binary data.

- CSV files are **text-based**
- Must open with `"r"` and encoding
- Binary files (images, PDFs) use `"rb"`
- Wrong mode leads to **errors or corrupted data**
- CSV parsing requires proper encoding handling



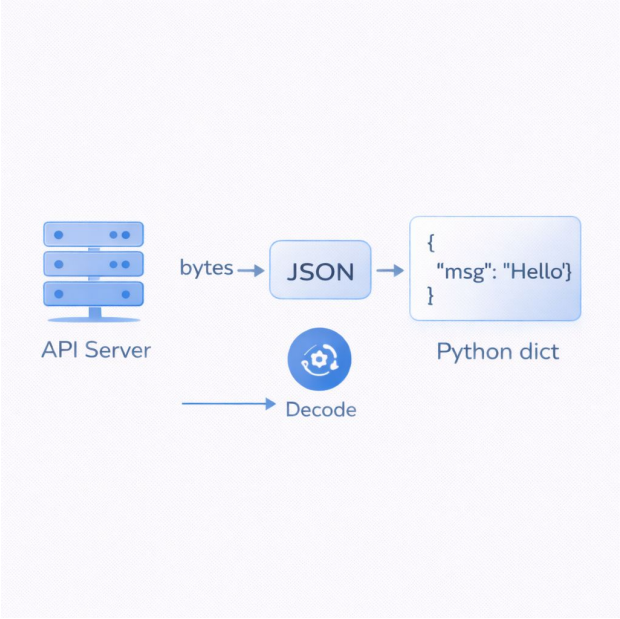
Demo:

```
open("data.csv", "r", encoding="utf-8")
open("image.png", "rb")
```

Handling API Responses

Working with text and bytes when consuming external APIs.

- API responses often return **JSON (text)**
- Data may arrive as **bytes over HTTP**
- Must decode using correct encoding (usually UTF-8)
- Libraries (e.g., requests) handle most conversions
- Always validate encoding when issues occur



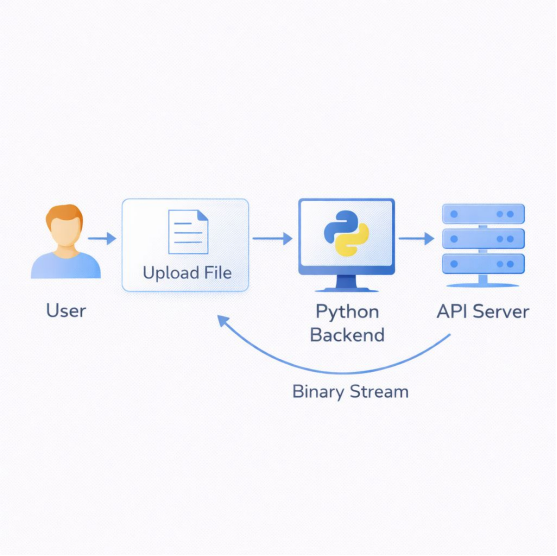
Demo:

```
import requests
res = requests.get("https://api.example.com")
data = res.json()
```

File Uploads

Handling binary data when uploading files to servers or APIs.

- File uploads are handled as **bytes**
- Common in web apps and APIs
- Requires `"rb"` mode when reading files
- Sent via HTTP multipart/form-data
- Important for images, PDFs, documents



Demo:

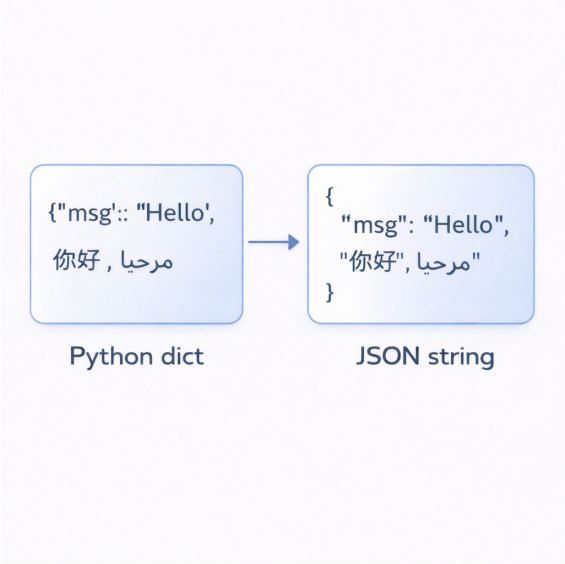
```
files = {"file": open("image.png", "rb")}
requests.post(url, files=files)
```

JSON Encoding Issues

Avoiding common errors when converting Python data to JSON.

- JSON requires **UTF-8 compatible text**
- Python `str` must be properly encoded
- Special characters may break serialization
- Use `json.dumps()` safely
- Ensure `ensure_ascii=False` for Unicode

```
import json
data = {"name": "José"}
json_str = json.dumps(data,
ensure_ascii=False)
```



Emoji & Multilingual Text

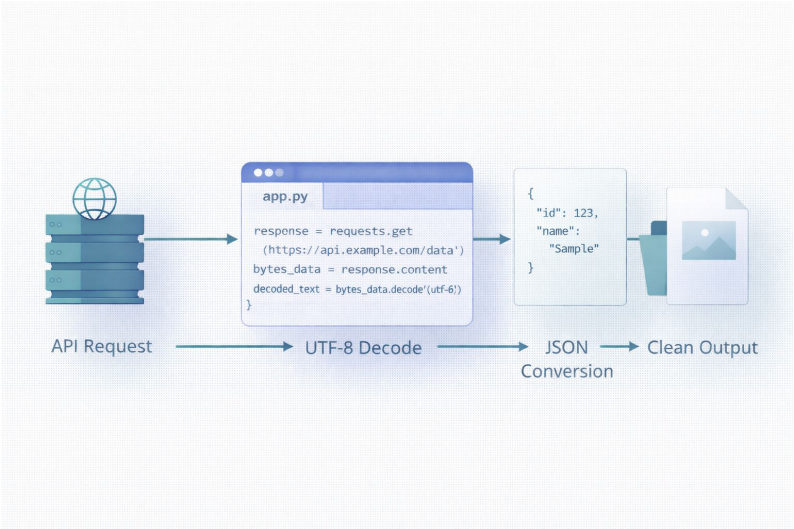
Handling global text correctly in modern applications.

- Unicode supports **emoji and multiple languages**
- UTF-8 is essential for proper display
- Critical for Malaysian systems (Malay, Chinese, Tamil)
- Improper encoding → broken text or errors
- Always test with real multilingual data

Demo:

```
text = "Hello 👋 你好 வணக்கம்"
print(text.encode("utf-8"))
```

Lab 3 — API Text Encoding

Build a simple Python script that:

- fetches data from a public API
- decodes the response manually
- converts the decoded text into JSON
- stores the result as clean UTF-8 output in a file